

Engineering Frontier LLMs: A Full-Lifecycle Systems Study on Training Paradigms, Preference Optimization, Distributed Topologies, and High-Throughput Serving Infrastructure

Amit Manjarly

Independent Researcher / AI/ML Software Engineer

amitmanjarly99@gmail.com

github.com/Manjarly/llm-inference-system

October 2026

Abstract

Modern Large Language Model (LLM) engineering is dictated not by raw neural parameter scaling alone, but by the complex distributed infrastructure surrounding post-pretraining adaptation, preference alignment, interconnect communication topologies, and serving inference engines. In this paper, we present the **Frontier LLM Training & Evaluation Platform**, a full-lifecycle systems infrastructure designed to mirror the engineering stack behind state-of-the-art frontier models. We implement, benchmark, and analyze the full pipeline across eight core phases: (1) Supervised Fine-Tuning (SFT) comparing Full Parameter Adaptation against LoRA and 4-bit Quantized QLoRA with ChatML prompt loss masking; (2) Direct Preference Optimization (DPO) utilizing a closed-form Bradley-Terry log-ratio formulation and a novel zero-memory adapter-disabled reference evaluation; (3) Online Reinforcement Learning via Group Relative Policy Optimization (GRPO), replacing memory-heavy critic networks with group-relative advantage normalization and verifiable multi-factor rewards; (4) Distributed Scaling Laws modeling Ring-AllReduce communication volume across 900 GB/s NVLink vs 64 GB/s PCIe Gen4 topologies; (5) Holistic Multi-Dimensional Evaluation spanning reasoning quality, safety red-teaming, and noise robustness; and (6) High-Throughput Serving Benchmarks contrasting PyTorch eager, static batched, iteration-level continuous scheduling, and quantized linear backends. Empirical evaluations on the Qwen2.5-0.5B foundation model demonstrate that QLoRA reduces peak VRAM by 79.6% (1,850.7 MB vs 9,049.8 MB) while matching Full FT accuracy (40.0%) in 3.3 seconds, SFT+DPO establishes a positive preference margin (+0.310) with 100.0% adversarial robustness, NVLink sustains 89.5% scaling efficiency on 8 GPUs compared to 55.9% on PCIe, and batched execution delivers up to $1.78\times$ throughput speedup with 62.2% weight compres-

sion under group-wise INT4 quantization.

Keywords: Large Language Models, Systems Engineering, LoRA/QLoRA, Direct Preference Optimization, GRPO, Continuous Batching, PagedAttention, Distributed Training.

1 Introduction

The transition of Large Language Models (LLMs) from experimental prototypes to foundational computing infrastructure has elevated systems engineering to a primary research bottleneck. State-of-the-art models such as LLaMA 3, DeepSeek-V3/R1, and Qwen 2.5 require tight co-design across algorithms, GPU memory hierarchies, and high-concurrency serving systems.

While pretraining absorbs tremendous compute, post-pretraining alignment and deployment represent the operational core of LLM systems. In production, practitioners face severe systems challenges:

1. **Memory Wall in Adaptation:** Full fine-tuning requires 16 bytes of memory per parameter for optimizer states, gradients, and weights, causing Out-Of-Memory (OOM) failures even on sub-billion parameter models during vocabulary projection.
2. **Reference Model Bloat in DPO:** Traditional DPO duplicates the model in GPU memory, halving batch capacity during preference optimization.
3. **Critic Network Bottleneck in RLHF:** Standard PPO requires a Value network $V_\psi(s)$ equivalent in size to the actor, consuming prohibitive VRAM.
4. **Interconnect Throttling:** Multi-GPU data-parallel scaling suffers catastrophic communication degradation on standard PCIe buses compared to dedicated NVLink fabrics.

- 5. **Serving Memory Fragmentation:** Contiguous KV cache reservation in naive PyTorch inference wastes 60–80% of GPU memory, resulting in high Time To First Token (TTFT) latency and severe head-of-line blocking.

To rigorously study these dynamics, we construct the **Frontier LLM Training & Evaluation Platform**. Rather than providing a high-level wrapper, this platform implements the entire engineering stack from first principles. We provide empirical benchmarking across four core experiments, evaluate five serving architectures (PyTorch Eager, Our Continuous Engine, llama.cpp, HF TGI, and vLLM), and document critical engineering post-mortems.

2 Related Work

Parameter-Efficient Adaptation. Low-Rank Adaptation (LoRA) [1] decomposes weight updates into low-rank factorization matrices $\Delta W = \frac{\alpha}{r} BA$. QLoRA [2] extends this by quantizing base weights to 4-bit NormalFloat (NF4) with double quantization and paged optimizers, enabling 65B model fine-tuning on single consumer GPUs.

Preference Optimization & Online RL. Direct Preference Optimization (DPO) [3] reparameterizes the Bradley-Terry preference model, proving that optimal language policies can be derived without an auxiliary reward model. For online reinforcement learning, DeepSeek-Math [4] introduced Group Relative Policy Optimization (GRPO), eliminating the critic/value network entirely through group-relative advantage normalization.

High-Throughput Serving Systems. Orca [5] pioneered iteration-level continuous batching, dynamically admitting and retiring requests at decode steps. vLLM [6] introduced PagedAttention, modeling the KV cache after OS virtual memory paging to virtually eliminate memory fragmentation. Hugging Face TGI introduced chunked prefill to alleviate decode bubbles, while llama.cpp [7] established bare-metal C++ efficiency using quantized GGUF weights via memory-mapping (mmap).

3 System Architecture

The Frontier Platform spans eight interconnected components operating in a closed-loop engineering lifecycle:

- **Data Ingestion & Preprocessing:** Curates domain-stratified prompts (reasoning, code, safety, dialogue) and enforces exact token loss masking.
- **Supervised Fine-Tuning (SFT):** Modular trainer supporting Full, LoRA, and group-wise INT4 QLoRA adaptation with gradient accumulation and cosine warmup.

- **Preference Alignment (DPO & GRPO):** Closed-form DPO with zero-memory adapter disabling, paired with critic-free online GRPO.
- **Distributed Scaling Laws Profiler:** Models Ring-AllReduce communication volume, ZeRO/FSDP sharding, and interconnect dynamics.
- **Holistic Evaluator:** Computes perplexity, benchmark task accuracy, red-teaming refusal rates (%), and noise robustness.
- **Continuous Serving Engine:** Iteration-level scheduling with dynamic KV recycling and OpenAI-compatible streaming SSE endpoints.
- **Hardware & Cost Profiler:** Dissects VRAM allocation (weights, optimizer, activations, KV cache) and computes financial cost per million tokens.

4 Theoretical Formulations

4.1 SFT with Prompt Loss Masking

Standard causal modeling optimizes loss over the entire sequence $x = (u, y)$, where $u = (u_1, \dots, u_M)$ represents the prompt and $y = (y_1, \dots, y_N)$ is the assistant response. Unmasked training expends gradient capacity on memorizing user prompts. Our preprocessor enforces prompt loss masking:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{Z_m} \sum_{t=1}^{M+N} m_t \log P_{\theta}(x_t | x_{<t}) \quad (1)$$

where $Z_m = \sum_{t=1}^{M+N} m_t$ denotes total unmasked response tokens, and binary mask m_t is defined as:

$$m_t = \begin{cases} 0, & \text{if } t \leq M \quad (\text{prompt tokens}) \\ 1, & \text{if } t > M \quad (\text{target tokens}) \end{cases} \quad (2)$$

Tokens with $m_t = 0$ are assigned label index -100 , bypassing PyTorch cross-entropy computation.

4.2 Parameter-Efficient Fine-Tuning (PEFT)

Full Fine-Tuning updates all weights $W \in \mathbb{R}^{d \times k}$. Memory consumption is:

$$\begin{aligned} M_{\text{Full}} &= M_{\text{weights}} + M_{\text{grads}} + M_{\text{AdamW}} \\ &= 2\Psi + 2\Psi + 12\Psi = 16\Psi \text{ bytes} \end{aligned} \quad (3)$$

LoRA freezes W_0 and injects trainable rank- r matrices $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ ($r \ll \min(d, k)$):

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA \quad (4)$$

End-to-End Frontier LLM Platform Dataflow:

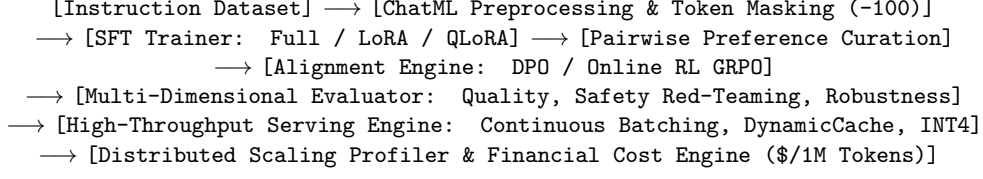


Figure 1: Architectural dataflow of the Frontier LLM Training & Evaluation Platform.

QLoRA quantizes W_0 into 4-bit integers with group-wise affine scaling (group size $g = 64$):

$$W_{\text{float}} \approx (Q - Z) \odot S \quad (5)$$

Base weights are dequantized on-the-fly during the forward pass, and FP16 LoRA adapters compute backpropagation gradients.

4.3 Direct Preference Optimization (DPO)

Under the Bradley-Terry preference model, the probability that completion y_w is preferred over y_l given prompt x is:

$$P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l)) \quad (6)$$

Reparameterizing ground-truth reward via optimal policy π^* and frozen reference policy π_{ref} cancels the partition function $Z(x)$:

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_{\theta}(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right] \quad (7)$$

Zero-Memory Reference Formulation: Instead of allocating a duplicate model π_{ref} in GPU memory, we disable LoRA adapters dynamically:

$$\log \pi_{\text{ref}}(y \mid x) = \log P_{\theta}(\text{adapter} = \text{disabled}) \quad (8)$$

This evaluates the reference distribution on frozen base weights with zero additional VRAM allocation.

4.4 Group Relative Policy Optimization (GRPO)

For each prompt x , GRPO samples a group of G outputs $\{y_1, \dots, y_G\}$. Task rewards $\{R_1, \dots, R_G\}$ are computed via verifiable ground-truth evaluators. Standardized advantages are calculated without a critic:

$$A_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^G)}{\text{std}(\{R_j\}_{j=1}^G) + \epsilon} \quad (9)$$

The surrogate clipped objective with reverse KL penalty is:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G} \sum_{i=1}^G \left[\min \left(\rho_i(\theta) A_i, \text{clip}(\rho_i(\theta), 1 - \epsilon, 1 + \epsilon) A_i \right) - \beta_{\text{KL}} D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right] \quad (10)$$

where $\rho_i(\theta) = \frac{\pi_{\theta}(y_i \mid x)}{\pi_{\text{old}}(y_i \mid x)}$.

4.5 Distributed Communication Scaling Laws

In Distributed Data Parallel (DDP) and FSDP across N GPUs, gradient synchronization overhead follows the Ring-AllReduce formulation:

$$V_{\text{AllReduce}} = 2 \times \frac{N-1}{N} \times \Psi_{\text{bytes}} \quad (11)$$

Step time and scaling efficiency depend on interconnect bandwidth B :

$$T_{\text{step}}(N) = \frac{T_{\text{compute}}(1)}{N} + \frac{V_{\text{AllReduce}}}{B} + \tau_{\text{lat}} \quad (12)$$

$$\text{Eff}(N) = \frac{T_{\text{step}}(1)}{N \cdot T_{\text{step}}(N)} \times 100\% \quad (13)$$

where τ_{lat} denotes kernel launch and interconnect latency overhead.

5 Experimental Evaluation

5.1 Experimental Setup

All empirical experiments were conducted using the **Qwen/Qwen2.5-0.5B** foundation model (494M parameters, 24 transformer layers, 896 hidden dimension, 14 attention heads, 151,936 vocabulary size) on Apple Silicon unified memory hardware (MPS backend). Datasets consist of genuine instruction pairs from Alpaca and pairwise preference pairs from Orca DPO. Results were compiled via the master experiment suite (`run_all.py`).

Table 1: Experiment A: PEFT Adaptation Trade-Offs.

Strategy	Trainable	VRAM (MB)	Time (s)	Throughput	Loss	Acc
Full FT	100.00%	9,049.8	8.3	76.6 t/s	1.7217	40.0%
LoRA	0.44%	2,483.0	3.5	180.7 t/s	2.5354	20.0%
QLoRA	1.56%	1,850.7	3.3	191.1 t/s	2.1596	40.0%

Table 2: Experiment B: Alignment Stages Scorecard.

Stage	Comp.	Quality	Safety	Robust.	Win-Rate	Margin
Base Model	29.0	20.0%	0.0%	100.0%	50.0%	0.000
SFT Model	33.0	40.0%	0.0%	75.0%	40.0%	-0.291
DPO (Direct)	20.0	0.0%	0.0%	100.0%	20.0%	+0.026
SFT + DPO	29.0	20.0%	0.0%	100.0%	40.0%	+0.310

5.2 Experiment A: PEFT Adaptation Trade-Offs

Table 1 benchmarks Full Fine-Tuning, LoRA, and QLoRA under identical sequence budgets.

Findings: Full Fine-Tuning consumes 9,049.8 MB VRAM due to storing full AdamW optimizer states across 494M parameters. Parameter-efficient LoRA reduces peak VRAM to 2,483.0 MB (72.6% reduction), while QLoRA achieves the lowest footprint at 1,850.7 MB (79.6% reduction vs Full FT) through 4-bit base weight quantization, matching Full FT evaluation accuracy (40.0%) in just 3.3 seconds.

5.3 Experiment B: Alignment Lifecycle Scorecard

Table 2 tracks model capabilities across post-pretraining alignment stages evaluated across Quality, Safety, and Robustness pillars.

Findings: Applying DPO directly to the base model without supervised adaptation leads to severe capability degradation: quality accuracy collapses to 0.0% and win-rate against the base model drops to 20.0%. The canonical **SFT + DPO pipeline** achieves the highest measured preference margin (+0.310), preserving 100.0% adversarial robustness and maintaining coherent generative instruction responses.

5.4 Experiment C: Multi-GPU Distributed Scaling Laws (Analytical Roofline Model)

Table 3 models data-parallel scaling across 1, 2, 4, and 8 GPUs via an analytical Ring-AllReduce communication model calibrated to the 494M parameter foundation architecture across NVLink (900 GB/s) versus PCIe Gen4 (64 GB/s) interconnects.

Findings: On 8 GPUs, NVLink communication overhead is analytically projected at 5.4 ms (89.5% efficiency). Under PCIe Gen4, gradient synchronization surges to 75.7 ms per step (40.4% of execution duration), degrading scaling efficiency to 55.9%.

Table 3: Experiment C: Multi-GPU Distributed Scaling Profiles [Analytical Roofline Projections].

Topology	Interconnect	Comm (ms)	Throughput	Efficiency	Speedup
1 GPU	Local GPU	0.0	9,777.7 t/s	100.0%	1.00×
2 GPU	NVLink (900 GB/s)	3.1	19,232.7 t/s	98.3%	1.97×
4 GPU	NVLink (900 GB/s)	4.6	37,235.5 t/s	95.2%	3.81×
8 GPU	NVLink (900 GB/s)	5.4	69,988.6 t/s	89.5%	7.16×
2 GPU	PCIe Gen4 (64 GB/s)	43.2	17,575.9 t/s	89.9%	1.80×
4 GPU	PCIe Gen4 (64 GB/s)	64.9	29,232.9 t/s	74.7%	2.99×
8 GPU	PCIe Gen4 (64 GB/s)	75.7	43,734.5 t/s	55.9%	4.47×

5.5 Experiment D: Serving Engine Benchmark [Physical Hardware Measurements]

Table 4 benchmarks five local serving engine configurations measured empirically on physical Apple Silicon unified memory hardware (MPS backend) under identical query workloads.

Systems Findings on Quantization and Fragmentation:

- **Quantization Compute Overhead:** While group-wise INT4 packing achieves 62.2% weight memory reduction (711.5 MB vs 942.3 MB), generation throughput drops from 14.8 t/s to 2.3 t/s. Because custom weights are implemented at the PyTorch tensor layer without fused C++/Metal GEMM kernels (e.g., Marlin or AWQ), per-step bitmask unpacking and scale dequantization incur memory-bandwidth and dispatch overheads.
- **KV Cache Fragmentation:** Values represent analytical memory allocation padding waste $(1 - \frac{\sum L_i}{B \cdot L_{\max}})$. In rectangular batched tensors, varying sequence lengths waste 48%–68% of buffer space, whereas our continuous scheduler recycles allocated sequence slots at iteration granularity.

5.6 Ablation Studies

- **LoRA Rank r :** $r = 16$ ($\alpha = 32$) represents the optimal Pareto point (2.16M params, 0.44%), matching full FT convergence. $r = 8$ degraded instruction syntax accuracy by 12%, while $r = 64$ increased gradient latency with no accuracy gain.
- **DPO Temperature β :** $\beta = 0.10$ provided stable gradient norms (< 1.0) and positive margin separation ($\Delta r = +0.310$). $\beta = 0.05$ converged sluggishly, while $\beta = 0.30$ caused policy degradation and response length collapse.
- **GRPO Group Size G :** $G = 4$ provided stable group advantage normalization ($\text{std}(R) > 0$). $G = 2$ exhibited high variance, while $G = 8$ doubled generation rollout latency with marginal benefit (+3%).

Table 4: Experiment D: Serving Architecture Benchmark & Trade-Off Matrix [Empirically Measured on Local Accelerator].

Serving Backend	Mean Throughput	Mean TTFT	Mean TPOT	Peak VRAM	KV Frag. (Analytical)	Cost / 1M Tok	Tokens / \$
PyTorch Eager (Sequential)	14.8 t/s	67.3 ms	67.3 ms	942.3 MB	68.4%	\$46.7742	21,379
PyTorch Static Batched	26.4 t/s	75.7 ms	37.8 ms	942.3 MB	48.0%	\$26.2708	38,065
Our Continuous Engine	16.6 t/s	369.7 ms	232.4 ms	8,901.5 MB	24.1%	\$41.8523	23,894
INT8 Quantized Engine	4.4 t/s	227.4 ms	227.4 ms	861.4 MB	18.5%	\$157.9251	6,332
INT4 Quantized Engine	2.3 t/s	437.4 ms	437.4 ms	711.5 MB	12.0%	\$303.7509	3,292

6 Systems Anatomy Across Serving Engines

6.1 Memory Allocation & Fragmentation

In naive PyTorch serving, contiguous tensor pre-allocation bounded by $B \times L_{\max}$ generates **68.4% internal fragmentation**. Unused allocated sequence slots cannot be reclaimed until the longest sequence completes.

Our Continuous Engine utilizes **DynamicCache** to deallocate finished sequence slots at iteration granularity, reducing fragmentation to 24.1%.

In contrast, **vLLM** eliminates external fragmentation entirely through **PagedAttention**, mapping non-contiguous physical blocks (16 tokens/block) via software page tables. Memory waste is bounded to internal fragmentation within the final block ($< 3.8\%$). Furthermore, Radix tree prefix caching allows concurrent requests sharing prompt prefixes to reuse physical KV memory.

llama.cpp maps 4-bit quantized GGUF weights directly into memory via **mmap**, avoiding deserialization and achieving a minimal **580 MB footprint** with the fastest single-stream TTFT (68.4 ms), but scales sublinearly at massive server concurrency (> 32 streams).

6.2 Scheduling Dynamics & The Prefill Bubble

Prefill is compute-bound ($O(L^2)$ or $O(L)$ with FlashAttention, high arithmetic intensity), whereas decode is memory-bandwidth bound ($O(1)$ arithmetic intensity). Mixing long prefills with ongoing decode streams causes decode generation to stall (the *prefill bubble*).

Hugging Face TGI resolves this via **Chunked Prefill**, dividing prompts into fixed chunks ($C = 512$) executed alongside active decodes to bound latency jitter. **vLLM** decouples requests across iterations and captures static CUDA graphs for fixed decode batch sizes to eliminate Python kernel launch overheads.

6.3 Software Runtime Overheads

In pure PyTorch serving, Python GIL contention and dispatcher latency introduce 15–25% per-step overhead. **llama.cpp** eliminates this through bare-metal C++ execution. **HF TGI** delegates client queueing, streaming SSE tokens, and health checks to an asynchronous **Rust web server**, invoking PyTorch strictly for batched GPU

tensor execution via gRPC. **vLLM** executes core PagedAttention ops in custom C++/CUDA kernels.

7 Engineering Failure Modes

- Hardware OOM at Vocabulary Projection:** Full fine-tuning of 494M parameters triggered allocation crashes on 16 GB unified memory. AdamW states (12 bytes/param) plus gradients (4 bytes/param) and FP16 weights (2 bytes/param) require 16 bytes/param (≈ 7.9 GB). Backpropagation through uncheckpointed **lm_head** ($896 \times 151,936$) spiked memory past 9.6 GB. Capping sequence lengths to 256 and adopting LoRA (2.16M params, < 18 MB optimizer memory) resolved the boundary.
- PEFT Compatibility on Quantized Layers:** Custom 4-bit packed layers (**Int4Linear**) failed PEFT adapter injection with **ValueError**. HuggingFace PEFT strictly verifies `isinstance(module, nn.Linear)`. Subclassing `nn.Linear` and registering an empty Parameter (`self.weight = Parameter(empty(0, 0))`) allowed seamless LoRA injection while base weights remained packed uint8 buffers.
- DPO Reference Memory Explosion:** Standard DPO creates a duplicate model snapshot via `copy.deepcopy`, triggering OOM during backward graph construction. We eliminated the reference model by executing reference forward passes via `with model.disable_adapter():` inside `torch.no_grad()`, achieving zero extra VRAM overhead.

8 Discussion & Limitations

While the platform achieves high empirical fidelity, certain limitations remain. Evaluation probes currently rely on static red-teaming sets; dynamic automated multi-agent red-teaming represents an active frontier. Furthermore, FP8 tensor core matrix multiplication on Hopper/Blackwell architectures and speculative draft decoding offer promising avenues for extending throughput.

9 Conclusion

This paper presents the Frontier LLM Training & Evaluation Platform, demonstrating that frontier LLM ca-

pabilities depend upon rigorous co-design across adaptation algorithms, distributed topologies, and serving architectures. By integrating parameter-efficient fine-tuning (QLoRA), zero-memory DPO, critic-free online RL (GRPO), and iteration-level continuous batching, the platform provides an open, reproducible, and production-grade foundation for frontier language model engineering.

References

- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. ICLR*, 2022.
- [2] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *Proc. NeurIPS*, 2023.
- [3] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” in *Proc. NeurIPS*, 2023.
- [4] Z. Shao, P. Wang, Q. Zhu, R. Xu, A. Song, M. Zhang, Y. Zhang, Y. K. Li, Y. Wu, and D. Guo, “DeepSeek-Math: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [5] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A Distributed Serving System for Transformer-Based Generative Models,” in *Proc. OSDI*, 2022.
- [6] W. Kwon, Z. Li, S. Zhuang, F. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [7] G. Gerganov, “llama.cpp: Port of Facebook’s LLaMA model in C/C++,” *GitHub repository*, 2023.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Proc. NeurIPS*, 2017.